Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 4

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ (Арифулов Р.Х.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

2026 г.

1 Цель работы

Изучение возможностей функционального программирования в языке Python.

2 Общее описание задания

Задание лабораторных работ № 3–4 состоит из решения нескольких задач. Задачи 1–4 (генератор field, генератор gen_random, итератор Unique, сортировка) выполнены в лабораторной работе № 3, в данной работе решены задачи 5–7.

Файлы, содержащие решения отдельных задач, располагаются в пакете lab_python_fp. Решение каждой задачи располагается в отдельном файле. При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

3 Задача 5 (файл print_result.py)

Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции. Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения. Если функция вернула список, то значения элементов списка должны выводиться в столбик. Если функция вернула словарь, то ключи и значения должны выводиться в столбик через знак равенства.

Файл print_result.py:

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)
        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            print(result)
        return result

    return wrapper
```

```
@print_result
def test_1():
    return 1


@print_result
def test_2():
    return 'iu5'


@print_result
def test_3():
    return {'a': 1, 'b': 2}


@print_result
def test_4():
    return [1, 2]


if __name__ == '__main__':
    print('!!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Результат выполнения программы приведен на рисунке 1.

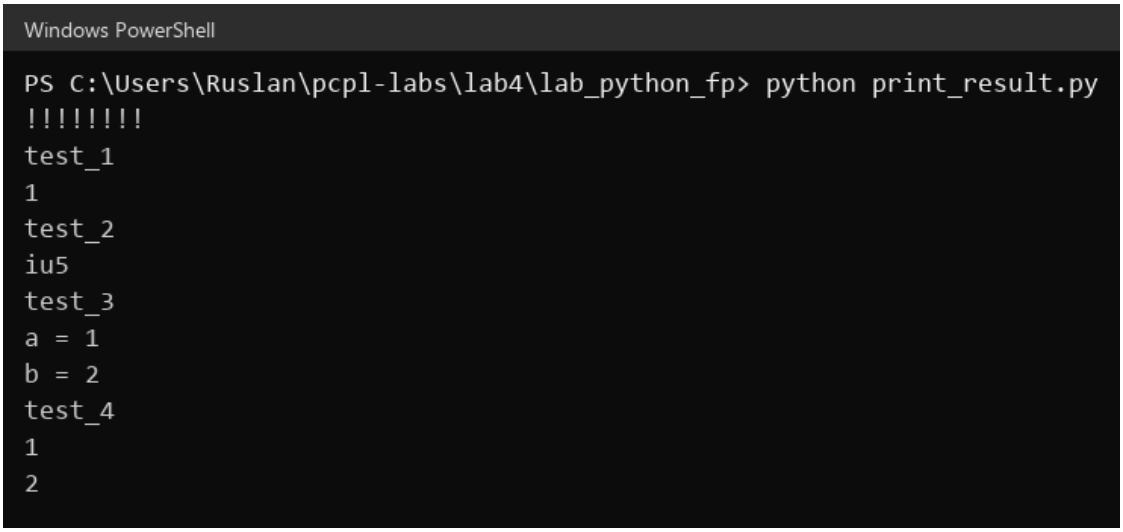


Рисунок 1 — Результат выполнения print_result.py

4 Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры cm_timer_1 и cm_timer_2, которые считают время работы блока кода и выводят его на экран. Менеджеры реализуют одинаковую функциональность, но реализованы двумя различными способами: на основе класса и с использованием библиотеки contextlib.

Файл cm_timer.py:

```
import time
from contextlib import contextmanager


class cm_timer_1:

    def __enter__(self):
        self.start = time.time()

    def __exit__(self, exc_type, exc_val, exc_tb):
        print('time:', round(time.time() - self.start, 3))


@contextmanager
def cm_timer_2():
    start = time.time()
    yield
    print('time:', round(time.time() - start, 3))


if __name__ == '__main__':
    with cm_timer_1():
        time.sleep(5.5)

    with cm_timer_2():
        time.sleep(2.5)
```

Результат выполнения программы приведен на рисунке 2.

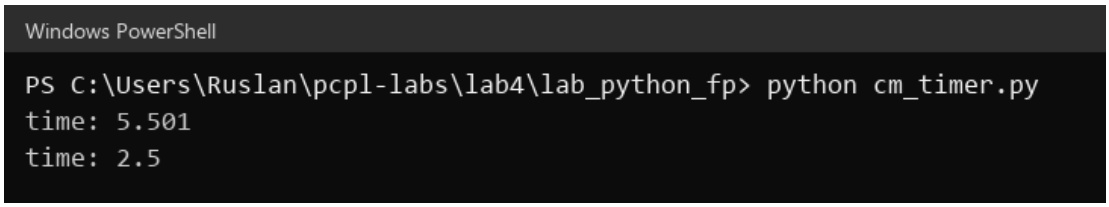


Рисунок 2 — Результат выполнения cm_timer.py

5 Задача 7 (файл process_data.py)

В файле data_light.json содержится фрагмент списка вакансий. Необходимо реализовать 4 функции f1, f2, f3, f4. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора print_result печатается результат, а контекстный менеджер cm_timer_1 выводит время работы цепочки функций.

– Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр.

– Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова «программист». Для фильтрации используется функция filter.

– Функция f3 должна модифицировать каждый элемент массива, добавив строку «с опытом Python». Для модификации используется функция map.

– Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Используется функция zip.

Функции f1, f2, f3 реализованы в одну строку, функция f4 — в две строки. Путь к файлу с данными передается при запуске программы.

Файл process_data.py:

```
import json
import sys

from field import field
from gen_random import gen_random
from unique import Unique
from print_result import print_result
from cm_timer import cm_timer_1

path = sys.argv[1]

with open(path, encoding='utf-8') as f:
    data = json.load(f)


@print_result
def f1(arg):
    return sorted(Unique(field(arg, 'job-name'), ignore_case=True),
key=str.lower)


@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))


@print_result
def f3(arg):
    return list(map(lambda x: x + ' с опытом Python', arg))


@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f'{name}, зарплата {salary} руб.' for name, salary in zip(arg,
salaries)]
```

```
if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Функция f1 выводит более 2000 профессий, поэтому на рисунке 3 приведено только начало вывода программы.

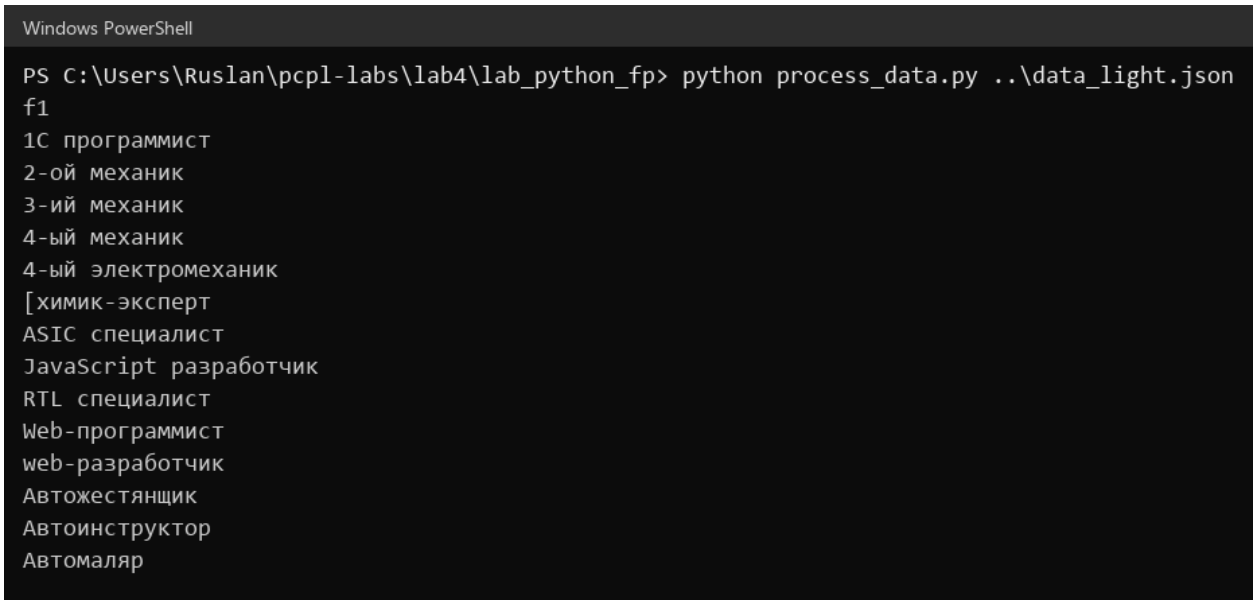


Рисунок 3 — Начало вывода process_data.py

Окончание вывода программы с результатами функций f2, f3, f4 и временем работы приведено на рисунке 4.

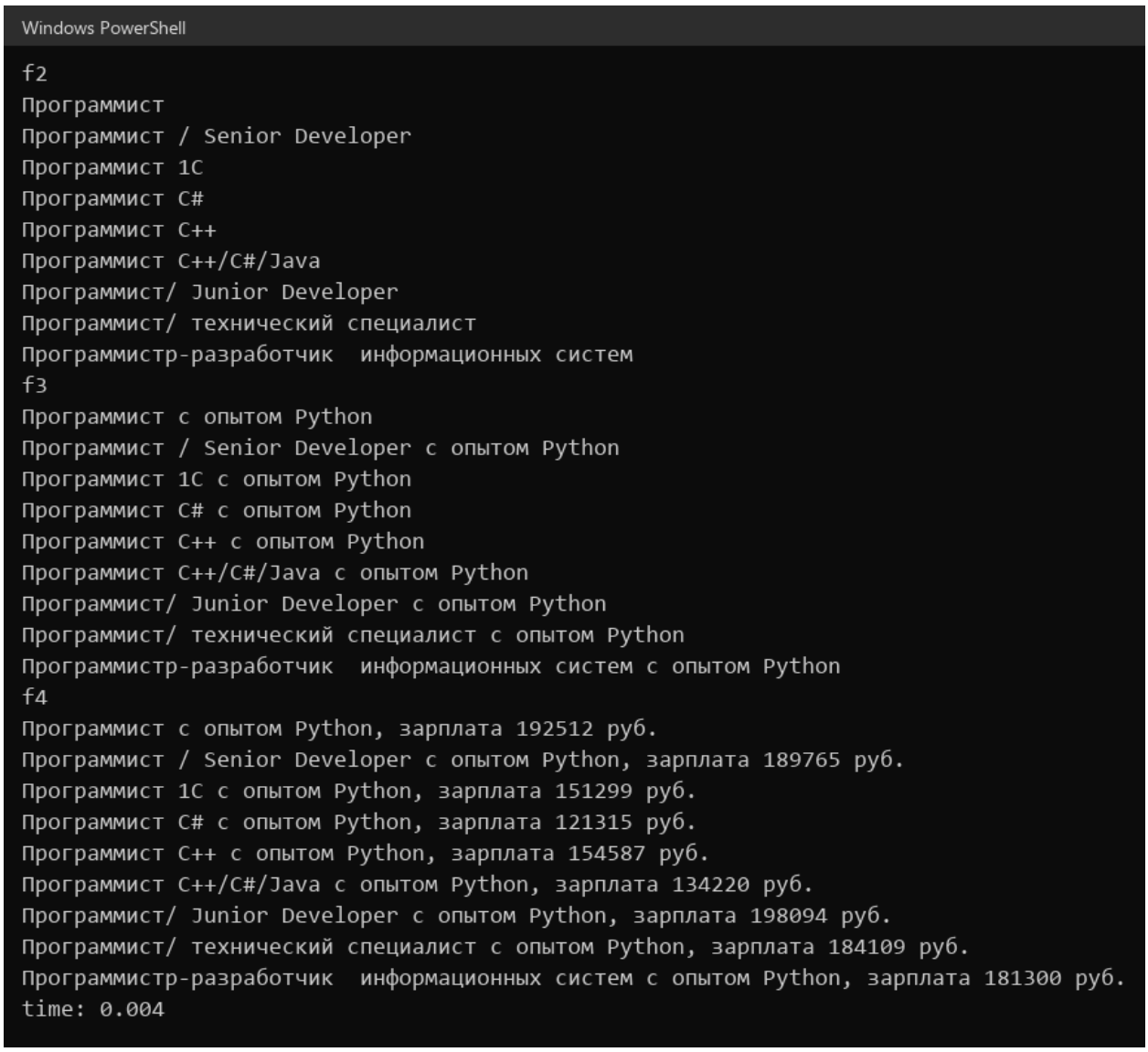


Рисунок 4 — Окончание вывода process_data.py